THOMAS MORE

# IQ Noodles Puzzle Assistant

Project Plan

Abdul Salam Aldabik
Student Bachelor Applied Computer Science

2025-2026

# Table of Contents



# 1. Introduction

This project plan sets out the goal, scope, requirements, approach, planning and risks for the IQ Noodles puzzle assistant, the system built during an internship for Smart NV, which ran from 23 February to 22 May 2026. It is the plan that guides the work and the reference the realization document later reports against. The plan opens with the client and the problem, then defines the scope and the requirements, describes the technical approach, lays out a phased planning with milestones, and closes with a risk analysis.

## 1.1. Client and context

Smart NV, which sells its games under the SmartGames brand, is a Belgian manufacturer of physical logic-puzzle games for children and families (SmartGames, 2024). Its games are entirely analog: when a player gets stuck on a challenge, the only help is a printed solution booklet, which needs reading and reveals the

full answer rather than a small hint. The company has no in-house computer-vision capability and no labelled training data, so this internship is a research-phase initiative that has to build the dataset, the model and the application from nothing. The wider team project covers three Smart NV games; this plan scopes the work to IQ Noodles, the game handled by the author.

## 1.2. Problem statement and goal

The goal is to turn the moment of getting stuck into a moment of progress. The system lets a player photograph a physical IQ Noodles board with a phone, reads which pieces are placed and where, and offers either the next piece to place or a complete solution, all without needing the player to read a booklet. Figure 1 shows a physical board, which is the input the system has to interpret.

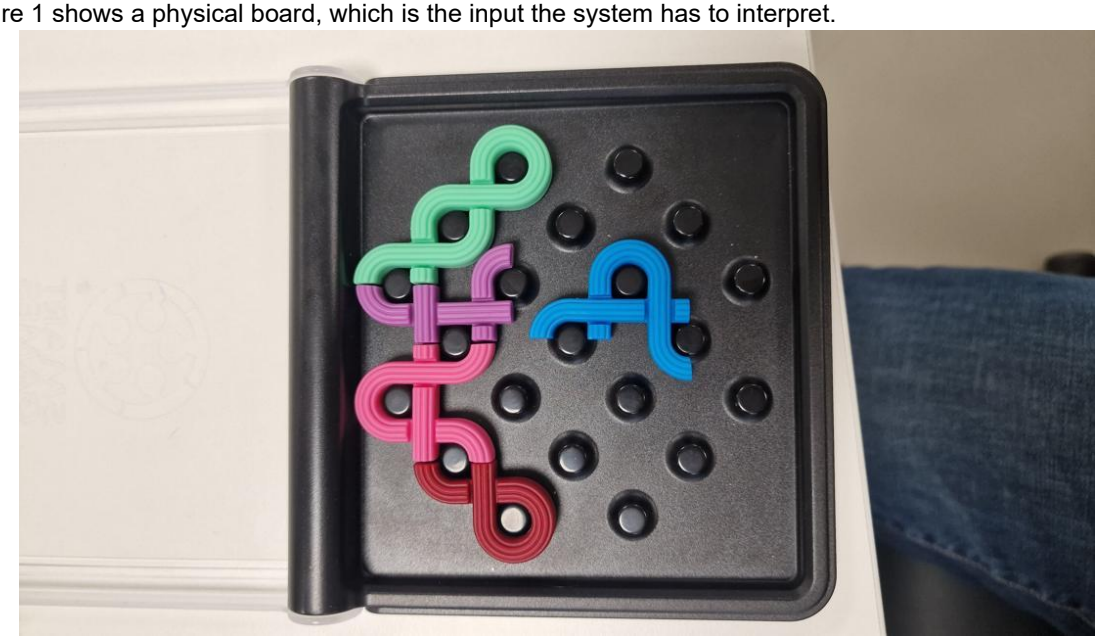

*Figure 1. A physical IQ Noodles board. The system must read the placed pieces from a single smartphone photo like this one and compute a hint.*

Three business goals sit behind this. The first is to reduce game abandonment by helping a child who is stuck. The second is to differentiate Smart NV products through AI-assisted play. The third is to build a reusable technical foundation, a dataset and a pipeline, that the company can extend to its other games later.

## 1.3. Structure of this document

The scope chapter states what is in and out of the deliverable. The requirements chapter lists the functional and non-functional requirements. The approach chapter explains the main technical choices. The planning chapter gives a phased schedule with milestones and deliverables, and the risk chapter lists the main risks and how they will be handled.

# 2. Project scope

This chapter fixes the boundary of the work so the plan stays realistic for one internship. It first describes the IQ Noodles game, then states what is in and out of scope.

## 2.1. The IQ Noodles game

IQ Noodles is a single-player puzzle played on a small board with eleven uniquely shaped, colour-coded "noodle" pieces (SmartGames, 2024). The pieces are curved paths that thread around the board's metal pins rather than rigid blocks, and every piece is unique in both shape and colour. This combination of shape and colour is what the computer-vision side has to recognise, and the curved geometry is what the solver has to reason about. Table 1 summarises the game.

*Table 1. Summary of the IQ Noodles game.*

| Property | Value |
|---|---|
| **Board** | 14 x 14 grid with 84 playable cells and 21 metal pins |
| **Pieces** | Eleven curved noodle pieces, each a unique shape and colour |
| **Placement** | Orthogonal paths around pins; no diagonal moves; no overlaps |
| **Player** | Single player, target age eight and above |

## 2.2. In scope

The deliverable is the complete IQ Noodles subsystem as a mobile-first Progressive Web App. The work in scope is the following.

- **Synthetic and real dataset.** A generated synthetic training dataset plus a set of real, annotated smartphone photos.
- **Trained detection model.** A YOLO segmentation model that detects the eleven pieces, the board and the pins, exported to run on the device.
- **Puzzle engine and solver.** A board model and a solver that completes any legal partial board and produces hints.
- **On-device vision pipeline.** The full path from a camera photo to a board state, running in the browser.
- **Application.** A mobile-first app with a scan mode and a manual placement mode, including the three-dimensional rendering of pieces.
- **Test and validation material.** Automated tests for the engine, solver and vision geometry, plus real-world scan validation.

## 2.3. Out of scope

To keep the deliverable achievable, the following are out of scope for this plan and are recorded as possible future work.

- **Cross-device dashboard.** Live progress sync to a second device is not part of the MVP.
- **Backend persistence.** No server-side storage of sessions or images; all processing is local.
- **Other Smart NV games.** IQ Puzzler Pro and IQ Waves are handled by other team members.
- **Extras.** Real-time video tracking, augmented-reality animations and social features are excluded.

# 3. Requirements

This chapter lists the requirements the result is measured against. Table 2 gives the functional requirements and Table 3 the non-functional requirements.

## 3.1. Functional requirements

*Table 2. Functional requirements for the IQ Noodles subsystem.*

| ID | Requirement |
|---|---|
| **FR-1** | The player can select the IQ Noodles board in the app and switch between scan and manual modes. |
| **FR-2** | The app captures a photo of the board with the phone camera in a mobile browser. |
| **FR-3** | A YOLO segmentation model detects the eleven pieces, the board outline and the pins on the device. |
| **FR-4** | The pipeline localizes the board, corrects perspective and maps detected pieces onto the digital board. |
| **FR-5** | A solver computes a valid completion from any legal partial board. |
| **FR-6** | The app generates a hint, either the next piece to place or a full solution. |
| **FR-7** | In manual mode the player places pieces by hand and can request validation and hints. |

## 3.2. Non-functional requirements

*Table 3. Non-functional requirements and their targets.*

| ID | Requirement | Target |
|---|---|---|
| **NFR-1** | Privacy (GDPR) | No photo of a child is transmitted or stored; only abstract board state is used. |
| **NFR-2** | Offline capability | The app functions without a network connection. |
| **NFR-3** | Cross-browser support | Runs in mobile Safari (iOS) and Chrome (Android). |
| **NFR-4** | Detection accuracy | mAP@0.5 of at least 0.75 across the key classes on real validation data. |
| **NFR-5** | Responsiveness | Detection and solving feel real-time on mid-range mobile hardware. |
| **NFR-6** | Maintainability | A modular architecture that can be retrained for other Smart NV games. |

# 4. Approach and methodology

This chapter explains the main technical choices and the reasoning behind them. Figure 2 shows the intended end-to-end flow, from a camera photo to a hint, with every stage running on the device.

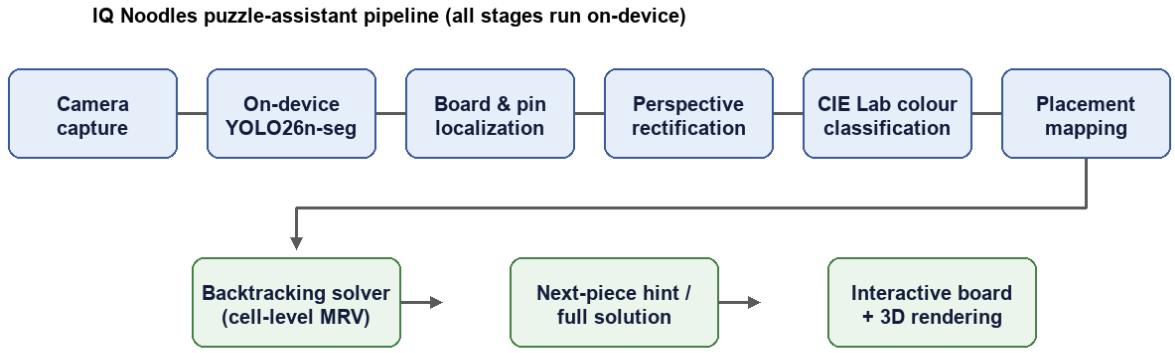


*Figure 2. Intended end-to-end flow. The camera frame passes through on-device detection, localization, rectification, colour classification and placement mapping, and the resulting board state feeds the solver, the hints and the interface.*

## 4.1. On-device computer vision

All image processing runs on the device. A nano-scale YOLO segmentation model runs in the browser through ONNX Runtime Web (Microsoft, 2024; Ultralytics, 2024), so photos never leave the phone, the app works offline, and the GDPR question for images of children is settled at the source. The alternative, uploading photos to a server, was rejected for exactly these reasons. The cost of the on-device choice is that the geometry and colour stages have to be written by hand in TypeScript, since the browser has no equivalent of OpenCV, and the plan accepts that cost.

## 4.2. Synthetic-to-real, two-phase training

Because no labelled dataset exists and real annotated photos are slow to produce, the model is trained in two phases. Phase one trains on a large synthetic dataset rendered in Blender, which produces high-quality images and exports the segmentation masks automatically. Phase two fine-tunes those weights on a smaller set of real photos annotated on Roboflow (Roboflow, 2026), so the model learns the appearance of the physical pieces under real lighting. The synthetic dataset is balanced across pieces and backgrounds so the model does not favour any one piece or learn a background by accident.

## 4.3. Solver

The puzzles are solved by algorithm rather than by a lookup table, because a player can start from a position no booklet contains. A backtracking search with a Minimum Remaining Values heuristic was chosen over Knuth's Algorithm X with Dancing Links (Russell & Norvig, 2020): it is easier to debug and supports partial boards and hints naturally, which the app needs. Algorithm X stays on file as a fallback if solver speed ever becomes a problem on a larger board.

## 4.4. Technology stack

The application is built with React 19, Vite and TypeScript (Vite, 2024), with the three-dimensional pieces rendered through React Three Fiber over Three.js (Pmndrs, 2024). The detection model is YOLO26-nano in its segmentation form, chosen for inference speed over the small accuracy gain of a heavier model. Synthetic data is rendered in Blender and real data is annotated on Roboflow. Table 4 summarises the stack and the reason for each choice.

*Table 4. Technology stack and the reason for each choice.*

| Area | Choice | Reason |
|---|---|---|
| **Detection model** | YOLO26n-seg (nano) | Fast on-device inference; segmentation gives piece shape, not just a box |
| **Inference runtime** | ONNX Runtime Web | Runs the model in the browser with a WebGPU and WebAssembly path |
| **Frontend** | React 19, Vite, TypeScript | Modular, typed components with fast builds |
| **3D rendering** | React Three Fiber, Three.js | Natural React fit; shared WebGL context for many previews |
| **Synthetic data** | Blender | High-quality renders with automatic mask export |
| **Real-data annotation** | Roboflow | Instance-segmentation labelling and dataset management |
| **Solver** | Backtracking with cell-level MRV | Simple, debuggable, natural support for partial boards |

# 5. Planning

This chapter sets out the schedule. The work is split into five phases across the internship. Table 5 gives the phases and Table 6 the milestones and deliverables. The phases overlap in practice, but each one has a clear centre of gravity.

## 5.1. Phased planning

*Table 5. Phased planning across the internship.*

| Phase | Weeks | Focus | Main outcome |
|---|---|---|---|
| **1. Foundation** | 1 to 2 | Onboarding, requirements, project charter, research | Agreed plan and approach |
| **2. Data and first model** | 2 to 5 | Synthetic data generation, first YOLO training | Working synthetic dataset and baseline model |
| **3. Engine and app** | 5 to 8 | Puzzle engine, solver, application skeleton | Solver and manual-mode app |
| **4. Vision and fine-tuning** | 8 to 11 | On-device pipeline, two-phase training, scan mode | End-to-end scan to hint |
| **5. Validation and handover** | 11 to 13 | Real-world testing, polish, documentation | Validated prototype and portfolio documents |

## 5.2. Milestones and deliverables

*Table 6. Key milestones and deliverables.*

| Deliverable | Description |
|---|---|
| **Synthetic dataset and generator** | A Blender generator and a balanced synthetic dataset with masks. |
| **Trained model** | A two-phase YOLO segmentation model exported to run on the device. |
| **Puzzle engine and solver** | Board model, placement generation, and a backtracking solver with hints. |
| **Vision pipeline** | On-device detection, localization, rectification and placement mapping. |
| **Application** | A mobile-first PWA with scan and manual modes and 3D rendering. |
| **Test and validation report** | Automated tests plus real-world scan validation results. |
| **Portfolio documents** | Project plan, realization document and reflection. |

# 6. Risk analysis

This chapter lists the main risks for the project and how each one will be handled. Table 7 gives the risks with an estimated impact and likelihood and a mitigation for each. The estimates use a simple low, medium and high scale.

*Table 7. Main project risks and their mitigations.*

| Risk | Impact | Likelihood | Mitigation |
|---|---|---|---|
| **No existing labelled dataset** | High | High | Generate synthetic data first; fine-tune on a small real set. |
| **Synthetic-to-real gap in colour and shape** | High | High | High-fidelity Blender renders; camera-calibrated colours; two-phase training. |
| **On-device performance and browser limits** | Medium | Medium | Nano model, ONNX runtime, one shared WebGL context. |
| **Placement accuracy from perspective distortion** | High | Medium | Pin-anchored homography instead of corner-only correction. |
| **Training infrastructure and time limits** | Medium | Medium | Self-hosted training with persistent sessions instead of a cloud notebook. |
| **Client dependencies (assets, references, feedback)** | Medium | Low | Coordinate early; agree what the company provides up front. |

# 7. Conclusion

This plan defines a focused, achievable internship deliverable: an on-device IQ Noodles puzzle assistant that reads a physical board from a phone photo and offers a hint, built from a synthetic-to-real training pipeline and a backtracking solver, all without any photo leaving the device. The scope is deliberately limited to the IQ Noodles subsystem, the requirements are concrete and measurable, and the main risks have clear mitigations. The phased planning leaves room at the end for validation and documentation, so the result reported in the realization document can be judged against the targets set here.

# REFERENCE LIST


Microsoft. (2024). ONNX Runtime Web (version 1.24.3) [Software]. https://onnxruntime.ai/

Pmndrs. (2024). React Three Fiber (version 9) [Software]. https://docs.pmnd.rs/react-three-fiber/

Roboflow. (2026). noodels dataset (version 4) [Data set]. https://roboflow.com/

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

SmartGames. (2024). IQ Noodles puzzle [Product page]. https://www.smartgames.eu/uk/one-player-games/iq-noodles

Ultralytics. (2024). YOLO documentation: Instance segmentation. https://docs.ultralytics.com/tasks/segment/

Vite. (2024). Vite documentation. https://vitejs.dev/


- Support for (internship) **documents on portfolio**:
  - General
    - Generative AI Policy
    - Information_management.pptx
    - Tips for writings in your portfolio.docx / Tips voor teksten in je portfolio.docx